SG6–HPC – TP2 MPI (mpi4py)

Compte Rendu de TP

Binôme :
Cêtre Bastien - Lutz Adrien

TP du : Vendredi 19 décembre 2025

Introduction

Ce compte rendu présente le travail réalisé lors du TP2 de SG6–HPC, consacré à la programmation parallèle avec `mpi4py`. L'objectif du TP est d'implanter et de comparer différentes stratégies de communications MPI pour le produit de matrices, puis d'analyser leurs performances sur un cluster de calcul.

1 Exercice – Communications point-à-point

1.1 Implémentation de l'algorithme en anneau (`Sendrecv_replace`)

1.1.1 Objectif

Implanter un produit de matrices distribué sur un anneau 1D de processus MPI en utilisant des communications point-à-point via la primitive `Sendrecv_replace`.

1.1.2 Principe et fonctionnement

Chaque processus MPI possède une tranche verticale des matrices A, B et C. Lorsque `RingSize = 1`, le produit matriciel est calculé localement à l'aide de `np.matmul`, sans communication.

Dans le cas général, l'algorithme repose sur la circulation des tranches de la matrice A sur un anneau logique de processus. À chaque itération, chaque processus calcule le produit entre sa tranche courante de A et sa tranche locale de B, puis stocke le résultat dans la partie correspondante de la matrice C.

La position du bloc à écrire est déterminée à partir de l'indice

$$owner = (Me - step) \bmod RingSize,$$

qui permet d'identifier l'origine de la tranche A utilisée à l'itération courante. La communication entre processus voisins est assurée par `Sendrecv_replace`, garantissant une circulation correcte des données sans allocation de buffers supplémentaires.

Après `RingSize` itérations, chaque processus a reçu toutes les tranches de la matrice A et a ainsi construit l'intégralité de sa tranche de la matrice résultat C.

Le code complet de cette implantation à base de communications point-à-point est fourni en annexe.

1.1.3 Validation fonctionnelle

Les résultats numériques obtenus sont identiques à ceux de la version séquentielle, confirmant la correction de l'implantation point-à-point.

```
sg6hpc1_14@kyle03:~/SG6-TP2$ /usr/bin/mpirun -np 4 python3 matprod.py --rp --s 1024
BENCHMARK: Configuration
  Matrix size: 1024 x 1024, kernel Id: 0
  Nb of processes: 4, nb of threads/process: 1
----
RING P2P: Initializations in progress
RING P2P: Distributed computation in progress
  Elpased time =  0.047 (s)
  Gflops       =  46.01 (Gflops)
  P:3: C[0][1023] = 1.4105835820e+14
  P:1: C[511][511] = -1.7904443742e+08
  P:0: C[1023][0] = -1.4050860239e+14
```

Figure 1 – Validation fonctionnelle du produit matriciel (communications point-à-point)

1.2 Comparaison des performances sur 4 et 8 nœuds

La méthodologie d'exécution batch et de mesure des performances est identique à celle utilisée pour les Exercices 2 et 3. Les tests ont été réalisés sur le cluster Kyle à l'aide des campagnes `RP2P-04-Kyle` (4 nœuds) et `RP2P-08-Kyle` (8 nœuds), pour des matrices de taille 16384×16384.

Différentes répartitions processus MPI / threads OpenBLAS ont été testées en faisant varier le nombre de processus par nœud (2, 4, 8 et 16) et le nombre de threads par processus (8, 4, 2 et 1). L'affectation des processus aux sockets et aux cœurs est contrôlée via les options `ppr` et `PE` de `mpirun`, afin de garantir une utilisation cohérente des ressources matérielles. Les performances sont exprimées en Gflops.

À titre d'exemple, la commande `/usr/bin/mpirun -np 8 -map-by ppr:1:socket:PE=8 -rank-by core python3 matprod.py -rp -s 16384 -nt 8` déploie un processus MPI par socket, chacun étant lié à 8 cœurs, tandis que l'option `-nt 8` configure l'utilisation de 8 threads OpenBLAS par processus.

1.2.1 Résultats expérimentaux

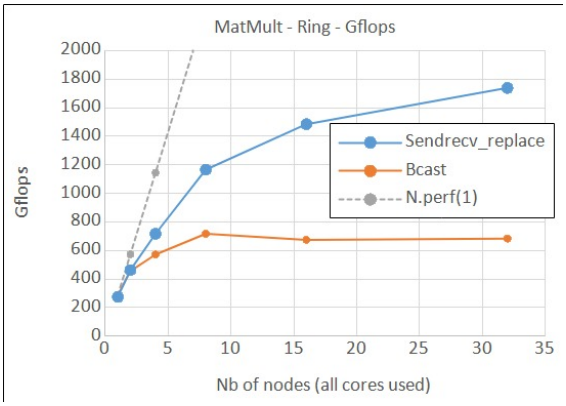


FIGURE 2 – Gflops

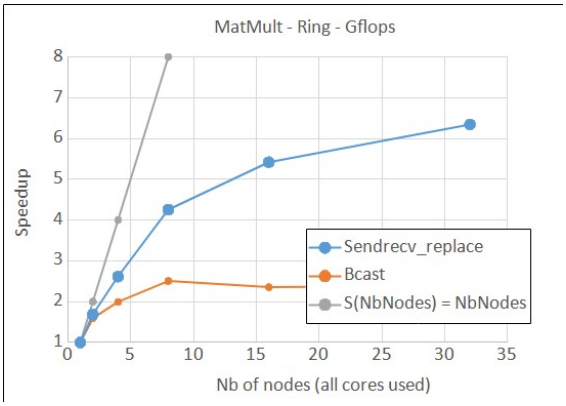


FIGURE 3 – Speedup

1.2.2 Analyse comparative

Le passage de 4 à 8 nœuds améliore les performances globales, mais le gain reste inférieur au facteur 2 attendu. Cette limitation s'explique par l'augmentation des communications inter-nœuds et les synchronisations imposées par l'algorithme en anneau.

Dans les deux cas, la meilleure configuration correspond à un compromis entre le nombre de processus MPI et le nombre de threads OpenBLAS par processus. Les tendances observées sont similaires sur 4 et 8 nœuds, indiquant que la scalabilité est principalement limitée par les communications.

1.3 Analyse globale

Les performances augmentent significativement de 1 à 8 nœuds, puis la croissance ralentit lorsque les coûts de communication deviennent dominants. Les accélérations observées restent éloignées de l'idéal $S(P) = P$, ce qui est cohérent avec un algorithme fortement communicant.

Dans ces conditions, l'utilisation d'un nombre toujours plus élevé de machines n'est pas pleinement justifiée. Les pertes de performances proviennent principalement des communications inter-nœuds, des synchronisations de l'anneau et du surcoût lié à la gestion conjointe des processus MPI et des threads OpenBLAS.

2 Exercice – Communications collectives

2.1 Complétion du code `mp_ring_col.py`

2.1.1 Objectif

Implanter le produit de matrices distribué sur un anneau 1D en utilisant des communications collectives MPI (`MPI_Bcast`) à la place des communications point-à-point.

2.1.2 Principe et fonctionnement

Chaque processus possède une tranche verticale des matrices A, B et C. Lorsque `RingSize = 1`, le produit matriciel est calculé localement à l'aide de `np.matmul` afin d'éviter toute communication inutile.

Dans le cas général, un tampon intermédiaire `AsCurrent` est utilisé pour stocker la tranche de la matrice A diffusée à l'étape courante. À l'itération `step`, le processus de rang `step` copie sa tranche locale dans ce tampon, qui est ensuite diffusé à tous les processus via `MPI_Bcast`.

Chaque processus calcule alors le produit local `AsCurrent`×`Bs` et stocke le résultat dans la tranche correspondante de la matrice C, délimitée par $[step \times \texttt{LocMatSide}, (step + 1) \times \texttt{LocMatSide})$.

L'utilisation du tampon `AsCurrent` garantit que la tranche locale `As` n'est jamais modifiée, ce qui évite toute restauration en fin de calcul.

Le code complet de l'implantation utilisant des communications collectives est présenté en annexe.

2.1.3 Validation fonctionnelle

Les résultats numériques obtenus avec cette version collective sont identiques à ceux des versions séquentielle et point-à-point, validant la correction de l'implantation.

```
sg6hpc1_14@kyle03:~/SG6-TP2$ /usr/bin/mpirun -np 4 python3 matprod.py --rc --s 1024
python3 matprod.py --rc --s 1024
BENCHMARK: Configuration
  Matrix size: 1024 x 1024, kernel Id: 0
  Nb of processes: 4, nb of threads/process: 1
----
RING COLLECTIVE: Initializations in progress
RING COLLECTIVE: Distributed computation in progress
  Elpased time =  0.051 (s)
  Gflops       =  42.52 (Gflops)
  P:3: C[0][1023] = 1.8343089493e+11
  P:1: C[511][511] = -1.7904443742e+08
  P:0: C[1023][0] = -2.8110865179e+14
```

FIGURE 4 – Validation fonctionnelle du produit matriciel (communications collectives)

2.2 Comparaison des performances sur 4 et 8 nœuds

La méthodologie d'exécution batch et de mesure des performances est identique à celle de l'Exercice 1. Les mêmes campagnes d'exécution, configurations MPI/OpenBLAS et métriques ont été utilisées, seul le schéma de communication collective différant de la version point-à-point.

2.2.1 Résultats expérimentaux

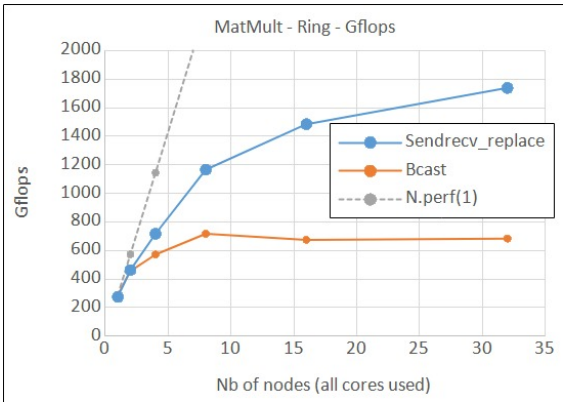


FIGURE 5 – Gflops

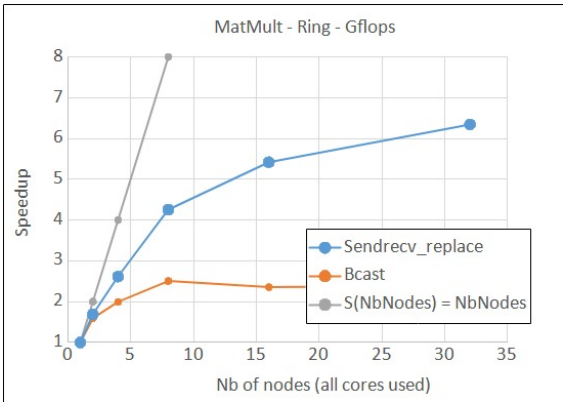


FIGURE 6 – Speedup

2.2.2 Analyse comparative

Le passage de 4 à 8 nœuds améliore les performances globales, mais le gain reste modéré en raison des synchronisations imposées par les communications collectives. Les performances restent du même ordre de grandeur que celles obtenues avec les communications point-à-point, avec une efficacité légèrement dégradée lorsque le nombre de processus augmente.

2.3 Analyse des performances mesurées

Les performances augmentent de manière significative entre 1 et 4 nœuds, puis la croissance des Gflops ralentit entre 4 et 8 nœuds. Au-delà, les gains deviennent marginaux, les coûts de communication collective et de synchronisation prenant le pas sur les bénéfices du parallélisme.

Les accélérations observées restent éloignées de l'idéal théorique $S(P) = P$. Cette perte d'efficacité est principalement due aux synchronisations globales induites par l'utilisation de `MPI_Bcast`, qui imposent une barrière à chaque itération de l'algorithme.

Dans ces conditions, l'utilisation d'un nombre toujours plus élevé de nœuds ne se justifie que jusqu'à un certain seuil. Au-delà, le coût des communications et des synchronisations limite fortement les gains de performance.

Comparées à la version à base de communications point-à-point, les performances de l'implantation collective sont du même ordre de grandeur. Toutefois, la version point-à-point présente généralement une légère meilleure efficacité, car elle évite certaines synchronisations globales imposées par les communications collectives.

3 Exercice 3 – Extraction des valeurs minimales par ligne

3.1 Complétion du code `mp_min_per_row.py`

3.1.1 Objectif

L'objectif de cet exercice est d'extraire la valeur minimale de chaque ligne de la matrice résultat C, une fois le produit matriciel distribué effectué, et de regrouper ces `MatSide` valeurs dans un tableau `numpy` sur le processus de rang 0.

3.1.2 Principe et fonctionnement

Chaque processus MPI possède une tranche verticale de la matrice résultat C, issue du produit matriciel distribué. Lorsque le programme s'exécute avec un seul processus, le calcul du minimum par ligne est effectué localement sans communication.

Dans le cas général, chaque processus commence par calculer localement le minimum de chacune des lignes de sa tranche C à l'aide de `np.amin(Cs, axis=1)`. Cette opération produit un tableau `numpy` de `MatSide` minimums locaux.

Une réduction collective est ensuite réalisée à l'aide de la primitive `MPI_Reduce` avec l'opérateur `MPI.MIN`. Cette opération permet de comparer les minimums locaux calculés par chaque processus et d'obtenir, sur le processus racine (rang 0), le tableau des minimums globaux, contenant une valeur minimale par ligne de la matrice C.

Cette approche repose sur une unique communication collective et garantit un coût négligeable par rapport au produit matriciel distribué.

Le code complet relatif à l'extraction distribuée des minimums par ligne est fourni en annexe.

3.2 Validation fonctionnelle

Le programme a été testé sur une demi-machine avec des matrices de taille 1024×1024 pour 1, 2 et 4 processus MPI, en utilisant l'option `-min` conjointement aux options `-rp` ou `-rc`, le calcul de la matrice C étant nécessaire avant l'extraction des minimums.

Quel que soit le nombre de processus utilisés, les mêmes valeurs minimales sont affichées par le processus de rang 0 :

- `MinValues[0] = -1.8316245948e+11`
- `MinValues[512] = -1.4078321186e+14`
- `MinValues[1023] = -2.8110865179e+14`

Ces résultats confirment la correction du calcul distribué des minimums par ligne et l'indépendance des résultats vis-à-vis du nombre de processus MPI utilisés.

```
sg6hpc1_14@kyle03:~/SG6-TP2$ mpirun -np 4 python3 matprod.py --s 1024 --rc --min
BENCHMARK: Configuration
  Matrix size: 1024 x 1024, kernel Id: 0
  Nb of processes: 4, nb of threads/process: 1
----
RING COLLECTIVE: Initializations in progress
RING COLLECTIVE: Distributed computation in progress
  Elpased time =  0.051 (s)
  Gflops       =  42.50 (Gflops)
  P:3: C[0][1023] = 1.8343089493e+11
  P:1: C[511][511] = -1.7904443742e+08
  P:0: C[1023][0] = -2.8110865179e+14
----
MIN VALUES: Distributed computation in progress
  P:0: MinValues[0] = -1.8316245948e+11
  P:0: MinValues[512] = -1.4078321186e+14
  P:0: MinValues[1023] = -2.8110865179e+14
```

FIGURE 7 – Validation fonctionnelle du calcul du minimum par ligne

Conclusion

Ce TP a permis d'implanter et de comparer plusieurs stratégies de calcul distribué pour le produit de matrices denses à l'aide de `mpi4py`. Les versions utilisant des communications point-à-point et collectives produisent des résultats numériques identiques, validant la correction des algorithmes.

Les exécutions batch réalisées sur 4 et 8 nœuds du cluster Kyle montrent une augmentation des performances avec le nombre de nœuds, mais des accélérations restant inférieures à l'idéal théorique $S(P) = P$. Cette limitation est principalement due au coût des communications inter-nœuds et aux synchronisations entre processus.

Les deux approches de communication présentent des performances globalement proches, la version point-à-point étant légèrement plus efficace lorsque l'échelle augmente. Enfin, l'extraction distribuée des minimums par ligne de la matrice C a un coût négligeable devant celui du produit matriciel et n'impacte pas significativement les performances globales.

A Annexes

Cette annexe contient les captures d'écran des codes développés ainsi que les tableaux de mesures de performances.

A.1 Code – Communications point-à-point

```python
# Matrix product computation on a 1D-ring of processes ---------------------------
def RingComputation(As, Bs, Cs, LocMatSide, MatSide, RingSize, Me, comm):

  if RingSize == 1:
    Cs = np.matmul(As, Bs)

  else:
    # On initialise l'indice de la tranche de A que l'on possède au début (étape 0)
    # Le processus 'Me' possède initialement la tranche de lignes 'Me'
    current_as_id = Me

    for step in range(RingSize):
      # Calcul des indices de lignes dans la matrice globale (et donc dans Cs)
      row_start = current_as_id * LocMatSide
      row_end = row_start + LocMatSide

      # Calcul du bloc de C et stockage au bon endroit dans la tranche verticale Cs
      Cs[row_start:row_end, :] = np.matmul(As, Bs)

      # Mise à jour de l'identifiant de la tranche As pour l'étape suivante
      # Si on envoie à Me-1 et reçoit de Me+1, les tranches défilent vers le haut
      current_as_id = (current_as_id + 1) % RingSize

      # Circulation des tranches As (Point-à-point)
      comm.Sendrecv_replace(
        As,
        dest=(Me - 1) % RingSize,
        source=(Me + 1) % RingSize
      )

  return Cs
```

FIGURE 8 – Implantation du produit matriciel avec communications point-à-point

A.2 Code – Communications collectives

```python
# Matrix product computation on a 1D-ring of processes ------------------------
def RingComputation(As,Bs,Cs,LocMatSide,MatSide,RingSize,Me,comm):

    if RingSize == 1: # Optimization for ring of 1 process
        Cs = np.matmul(As,Bs)

    else:            # Generic case (more than 1 process)
        # Buffer de la tranche A à utiliser à l'étape courante
        AsCurrent = np.zeros(As.shape, np.float64)

        # Boucle de calcul-communication sur l'anneau (taille RingSize)
        for step in range(RingSize):
            # - Préparer le buffer de la tranche A à diffuser
            # A l'étape 'step', le processus de rang 'step' diffuse sa tranche initiale
            if Me == step:
                AsCurrent[:] = As[:]

            # - Communication collective de la tranche courante
            comm.Bcast(AsCurrent, root=step)

            # - Calcul de l'intervalle de lignes pour stocker les résultats dans Cs[row1:row2,:]
            row1 = step * LocMatSide
            row2 = row1 + LocMatSide

            # - Stocker le produit numpy AsCurrent x Bs dans Cs[row1:row2,:]
            Cs[row1:row2, :] = np.matmul(AsCurrent, Bs)

        # Rmq: Pas besoin de restaurer As ici car nous avons utilisé AsCurrent pour le Bcast.
        # Ainsi, As de chaque processus est resté dans son état initial[cite: 145].

    return Cs
```

FIGURE 9 – Implantation du produit matriciel avec communications collectives

A.3 Code – Extraction des minimums par ligne

```python
import os
import sys
import time
import math

import numpy as np

import mpi4py
from mpi4py import MPI


# Matrix product computation on a 1D-ring of processes ---------------------------
def minvalues(Cs, LocMatSide, MatSide, RingSize, Me, comm):
    # 1. Calcul des minimums locaux pour chaque ligne de la tranche Cs
    # axis=1 permet de calculer le minimum sur les colonnes pour chaque ligne
    local_mins = np.amin(Cs, axis=1)

    # 2. Préparation du tableau de réception sur le processus 0 uniquement
    if Me == 0:
        global_mins = np.empty(MatSide, dtype=np.float64)
    else:
        global_mins = None

    # 3. Réduction collective pour trouver le minimum global de chaque ligne
    # On utilise MPI.MIN pour comparer les vecteurs de chaque processus
    comm.Reduce(local_mins, global_mins, op=MPI.MIN, root=0)

    return global_mins
```

FIGURE 10 – Implantation du calcul distribué des minimums par ligne

A.4 Tableaux de résultats de performances

Sendrecv_replace							
	Nb of nodes	1	2	4	8	16	32
	1pr/socket 8th/pr	269,69	444,89	716,30	991,90	1246,44	1306,51
	2pr/socket 4th/pr	273,59	**462,50**	712,68	1027,53	1181,81	1660,56
	4pr/socket 2th/pr	**273,85**	459,23	710,47	962,63	**1484,26**	**1738,33**
	8pr/socket 1th/pr	268,88	**451,14**	696,93	1165,85	1452,38	1634,93
		q					
	Nb of nodes	1	2	4	8	16	32
	Best Gflops	*273,85*	*462,50*	*716,30*	*1165,85*	*1484,26*	*1738,33*
	Speedup	*1,00*	*1,69*	*2,62*	*4,26*	*5,42*	*6,35*
Bcast							
	Nb of nodes	1	2	4	8	16	32
	1pr/socket 8th/pr	**286,06**	**455,39**	489,00	716,67	**674,01**	**682,77**
	2pr/socket 4th/pr	274,03	434,12	469,81	674,32	606,47	554,46
	4pr/socket 2th/pr	267,23	313,87	571,09	596,65	573,37	480,03
	8pr/socket 1th/pr	274,46	419,25	534,72	574,89	553,95	520,73
		Gflops					
	Nb of nodes	1	2	4	8	16	32
	Best perf	*286,06*	*455,39*	*571,09*	*716,67*	*674,01*	*682,77*
	Speedup	*1,00*	*1,59*	*2,00*	*2,51*	*2,36*	*2,39*
ideal	NbNodes * Gflops(1)	286,06	572,12	1144,24	2288,48	4576,96	9153,92

FIGURE 11 – Tableau récapitulatif des performances mesurées